SLIIT

COMPUTING | BUSINESS | ENGINEERING

IE3090
Network Programming
3rd Year, Semester I

Structured Reflection — IE3090 RemoteOps Assignment

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

5th October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number : IT24101899

Name : Kavirathna N R

I used Claude, DeepSeek and ChatGPT at different phases while working on Part 1. The AI helped me understand the socket API, write the initial boilerplate code for the Agent and Controller, debug the “missing” #include and the Makefile “missing separator” errors, and grasp the personalisation requirements described in the brief. I captured all meaningful interactions with the AI in the attached prompt log.

The AI was genuinely helpful in clarifying fundamental concepts to me. The walk-through of the socket(), bind(), listen(), and accept() functions provided me with the theoretical basis to implement my TCP server correctly. The AI correctly identified the error messages I received and recommended using tmux to keep my server processes running after disconnection.

On the other hand, the AI misled me and provided inaccurate information several times. It suggested using `stty -echo` to capture a clean screenshot, but this left my terminal unusable, requiring me to type `reset` to fix it. The skeleton server code the AI generated omitted the #include header, rendering it non-compileable until I identified the error. The AI recommended using a 2-second interval for UDP monitoring, which resulted in the Controller’s terminal being overwhelmed with traffic, making it impossible to capture a screenshot. I had to increase the interval to 5 seconds (MON_INTERVAL_SEC) to make it work.

Aside from mechanical edits and changes to tailor the code to my requirements (using my port, SID, token, log file name, storage directory path, etc.), I substantially rewrote sections of the code to use different function names (logmsg, reply, do_sysinfo, recv_bytes) and justify my design decisions better. The concurrency rationale, for instance, involved substantial rewriting to explain why threads were a better choice than forking or using select(). Not a single sentence in my report was taken verbatim from any of the AIs.

The main thing I learned was that a single-byte difference can derail the entire communication. I had to think through the buffering and framing implications of every PUT and GET before implementing them. I have acquired a solid

conceptual understanding of how a TCP server works, including how it serves multiple clients concurrently (one thread per connection), why line-based framing and buffering (monitoring for \n) are appropriate, and why leftover bytes should be kept for subsequent commands. I learned that SIGPIPE kills a process attempting to write to a closed TCP connection and that the MSG_NOSIGNAL flag and(SIGPIPE) are viable solutions. Moreover, I have gained an appreciation for the differences between UDP and TCP, including the best-effort nature of UDP. The Controller does not need to handle lost datagrams during monitoring since they only carry statistical information.

What surprised me most was that there was much more “glue” than I anticipated. Every response had to have the correct SID, and error codes had to be consistent. Even something as simple as an extra \r can break things. I was not expecting to handle disconnects so cleanly in the Agent — I got away with simply cleaning up in the client_thread() function.

In future assignments, I intend to test every implemented command as I go instead of leaving it all for the end. I would also keep a design diary for the same purpose. If I had used tmux from the beginning, I would not have struggled to resume my work after my SSH session died. I can confidently say that I fully understand all the functions in my code. I know why I chose port 9410 and what the 9981 in my SID means. Finally, I am prepared to customise my code further if the Lab Assessment requires.